

Object Stereotypes

Objects come in kinds. Three vocabularies for saying which kind you have.

What Kind of Object Is This?

Cohesion told us to keep together what changes together. It did not tell us what sorts of things there are to keep together.

The question

Two classes, both cohesive, both well named — and still not the same kind of thing. What is the difference?

The answer

Three people answered it, thirty years apart, and each sorted objects by a different question.

Why today

Because the kind decides what a test must assert — which is the next session's subject.

Learning Objectives

- Name the kind of an object, and say what question that answer came from.
- Tell an entity from a value object by asking about identity, not fields.
- Recognize a repository, a factory and a domain service on sight.
- Know why the same word means different things in different books.

The System We'll Use

Every example today comes from one small system — a community library. Here it is, before we say a word about kinds of objects.

What it does

- A member borrows a copy
- ...and returns it, late or on time
- ...reserves a title when none are free
- ...renews a loan nobody is waiting for

What is in it

- Books, and the copies of them
- Members, each on a tier
- Loans, with a due date
- Reservations, and overdue fines

That is the whole system. Today we ask what kind each of those things is — three times, and three different answers.

01 · 1992

Jacobson

The first claim that objects come in kinds.

Entity · Boundary · Control

Ivar Jacobson sorted objects by one question: what makes this change?

Kind	What it is	What changes it
Entity	A domain concept — data and its rules	The business changing its mind
Boundary	Everything that touches the outside	A new GUI screen, web page or device
Control	The steps of one use case	The procedure changing

The control object is the glue. It carries the steps of one use case — the behavior that belongs to no single entity — so the boundary never reaches into the model, and the model never learns there is a web page. Three kinds, three reasons to change: the single responsibility principle, eleven years early.

 **Object-Oriented Software Engineering · Jacobson et al., 1992** — *where the three kinds were first named*

The Notation Died, the Idea Did Not



You have met them already

- Boundary — a controller, a CLI, an adapter
- Control — a service holding one use case
- Entity — the objects the rules act on



Why it still holds

- Each kind changes for its own reason
- Mixing them is how a god class starts
- Every architecture since re-draws this line

02 · 2002

Wirfs-Brock

Sorted by responsibility, not by layer.

Six Roles an Object Can Play

Wirfs-Brock asked a different question: what is it responsible for?

Role stereotype	It exists to...	In our code
Information holder	know something, and say so	Book, Loan
Structurer	hold a relationship together	the reservation queue
Service provider	do work on request	LoanService
Coordinator	delegate, in the right order	returnBook()
Controller	decide what happens next	the borrowing rules
Interfacer	translate between two worlds	the repository adapters

An object may play two roles. Three is usually a class asking to be split.

Telling the Middle Four Apart

Information holder and interfacer are easy to spot. These four blur into each other, and the line between them is worth learning.

Role	The line that separates it	In our code
Structurer	knows what is related to what, not how to compute	the reservation queue
Service provider	knows how to do the work, and does it itself	FinePolicy
Coordinator	knows who acts next; sequences without deciding	returnBook()
Controller	knows what should happen; the decision lives here	the borrowing rules

Coordinator or controller? If the decision can move out and the object still works, it was coordinating.

The Question That Picks the Role

Responsibility-driven design starts by asking what an object owes its neighbors — not what fields it has.



Asking about data

- "What does a Loan hold?"
- isbn, member, dates, renewals
- You get an information holder, always



Asking about responsibility

- "What does a Loan owe its caller?"
- Is it overdue? How late? Renew itself.
- You get behavior, and the fields follow

 **Object Design: Roles, Responsibilities, and Collaborations** · Wirfs-Brock & McKean, 2002 — *responsibility-driven design starts in her 1990 book; the stereotypes are here*

03 · 2003

Evans

Sorted by identity and lifecycle.

The Identity Question

Eric Evans' test is one sentence: if two of them have the same values, are they the same thing?

Value object — no identity

- 20 TL is 20 TL. There is no other one.
- Immutable; equal by its values
- Money, Isbn, MemberId

Entity — identity that survives change

- Ayşe upgrades tier; still the same member
- Equal by id, whatever else changed
- Member, Book, Loan

A record gives you value semantics. Wrong for an entity — two versions of the same member stop being equal. Right for a DTO, which has no identity to lose. (DTO: Fowler, PoEAA 2002.)

Where Objects Live, and Who Makes Them

Stereotype	What it is for	In our code
Aggregate	One entry point to a cluster of objects, so its rules hold	a title's queue and hold
Repository	Where entities are kept — an interface the domain owns	LoanRepository
Factory	Builds a valid object, so nothing half-made exists	Loan.starting(...)
Domain service	Behavior belonging to no single entity	LoanService
Application service	Sequences one use case; owns no rules itself	borrow(), end to end

A domain service decides; an application service only sequences. And it is a last resort — behavior that fits inside an entity belongs there; services that swallow every rule leave an anemic model.

 **Domain-Driven Design · Eric Evans, 2003** — *Part II is the whole vocabulary, one chapter per stereotype*

Not a Rename — a Different Cut

Ask Evans' question of each of her roles, and see where they land.

Wirfs-Brock role	Evans asks instead	Where it lands
Information holder	Does it have identity?	Entity or value object — a split
Structurer	What must stay consistent?	Aggregate root, or repository
Service provider	Can the object do it itself?	The entity first; else a service
Coordinator	Who sequences the use case?	An application service
Controller	Is it a rule about objects?	The entity, or a specification
Interfacer	—	Outside the set — an adapter

Not a rename. LoanRules is a fine service provider to Wirfs-Brock, and an anemic model to Evans.

Three Vocabularies, One Question

They do not disagree. Each sorts the same objects by a different question, and each question is worth asking.

	Sorts objects by...	Best when you are asking
Jacobson · 1992	what makes it change	where the seams of a system go
Wirfs-Brock · 2002	what it is responsible for	which object should own a job
Evans · 2003	whether it has identity	how to model a domain, and test it

This is why the same word means different things. Entity is Jacobson's long-lived information, and Evans' object-with-identity. Module 13 — Hexagonal, Onion & Clean Architecture — uses Entities for something else again: enterprise rules. Always ask whose vocabulary is being spoken.

The Library Loan Service, Labeled

The project you will build has one of nearly everything. Naming the kind is how you know where a new rule belongs.

Values and entities

- Money · Isbn · MemberId — value objects
- MembershipTier — a value object, not a type
- Book · Member · Loan — entities
- Hold — an entity with a deadline

Everything that acts

- LoanRepository — a repository
- Loan.starting(...) — a factory
- LoanService — a domain service
- FinePolicy — a policy, swappable

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 14 tests here

`stereotypes/MemberAsValue`
an entity written as a record — the smell

`stereotypes/Member · MemberCard`
identity, and the DTO it hands out

`stereotypes/Loan · LoanRules`
rules with the data, or beside it

C# / .NET

xUnit · 14 tests here

`Stereotypes.cs`
members, cards, loans

`StereotypesTests.cs`
identity · factory & DTO · anemic

Python

pytest · 15 tests here

`stereotypes.py`
members, cards, loans

`tests/test_stereotypes.py`
identity · factory & DTO · anemic

github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

The smell is not a bug. Both models give the same answer — run them and see. What differs is where the next rule has to go.

*Clone one, run the suite, then break something and watch which test goes red. **Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Objects come in kinds; naming the kind is a design decision, not paperwork.
- ✓ Jacobson sorts by what changes it, Wirfs-Brock by what it owes, Evans by identity.
- ✓ Entity or value object? Ask whether two with the same values are the same thing.
- ✓ Reach for a domain service last — behavior belongs in the model where it fits.
- ✓ The kind of an object decides what its test must assert. That is next.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Object-Oriented Software Engineering: A Use Case Driven Approach** Ivar Jacobson et al. · Addison-Wesley, 1992

Where entity, boundary and control come from — objects sorted by what changes them.

📖 **Object Design: Roles, Responsibilities, and Collaborations** Wirfs-Brock & McKean · Addison-Wesley, 2002

The six role stereotypes, and how to pick one by asking what an object is for.

📖 **Domain-Driven Design** Eric Evans · Addison-Wesley, 2003

Entity, value object, aggregate, repository, factory, domain service — the set this course uses.

📖 **Implementing Domain-Driven Design** Vaughn Vernon · Addison-Wesley, 2013

Evans' vocabulary in working code, when the original book stays too abstract.

Wirfs-Brock — “Characterizing Your Objects” on role stereotypes → wirfs-brock.com/PDFs/Characterizing_Objects.pdf

Next session

What's Next

Module 8

Developer Testing

with Akin Kaldıroğlu

Unit and integration testing — and a different thing to assert for each kind of object we just named.

Name the Kind.

What changes it · what it owes · whether it has identity.